OS-Lab1

Part A

实现思路

Part A 要添加一个 `trace(mask)` 系统调用。掩码决定跟踪哪些调用，内核在这些调用返回前打印进程 PID、调用名称和返回值。设置只对当前进程及其之后创建的子进程生效。

掩码的第 `n` 位对应编号为 `n` 的系统调用。比如 `SYS_read` 是 `5`，`1 << SYS_read` 就是 `32`，所以 `trace(32)` 只跟踪 `read`。如果要跟踪多个调用，可以把对应的位做按位或运算；设置为 `0` 则关闭跟踪。

实验已经提供了 `user/trace.c`。它先调用 `trace(mask)`，再用 `exec()` 运行指定命令。可以把掩码放在 `struct proc` 中，让每个进程保存自己的设置。`exec()` 替换程序时保留这个字段，`fork()` 创建子进程时复制它。

打印代码放在 `syscall()` 中：先调用处理函数，拿到返回值后再检查掩码。所有系统调用都经过这里，只需在这一处添加判断。

关键代码

补齐用户态接口和系统调用编号

在 `user/user.h` 中声明接口：

```
int trace(int);
```

在 `user/usys.pl` 中注册汇编入口：

```
entry("trace");
```

在 `kernel/syscall.h` 中为新系统调用分配编号：

```
#define SYS_trace  22
```

编译时，`user/usys.pl` 自动生成 `user/usys.S`，其中 `trace` 的入口相当于：

```
trace:
  li a7, SYS_trace
  ecall
  ret
```

调用时，参数放在 `a0`，系统调用编号放在 `a7`。执行 `ecall` 后进入内核。还需要把 `$U/_trace` 加入 `Makefile` 的 `UPROGS`，这样 xv6 文件系统里才有可执行的 `trace` 命令。

保存进程的跟踪设置并处理继承

在 `kernel/proc.h` 的 `struct proc` 中新增字段：

```
uint tracemask;              // System calls to trace (one bit per syscall)
```

在 `kernel/proc.c` 的 `allocproc()` 中初始化：

```
p->tracemask = 0;
```

进程表项会被重复使用，所以每次分配时都要清零掩码，否则新进程可能沿用上一个进程的跟踪设置。

在 `fork()` 中复制父进程的掩码：

```
np->tracemask = p->tracemask;
```

`p` 是父进程，`np` 是子进程。这里复制的是掩码数值，子进程之后修改自己的掩码不会影响父进程。

实现内核处理函数

在 `kernel/sysproc.c` 中新增：

```
uint64
sys_trace(void)
{
  int mask;

  argint(0, &mask);
  myproc()->tracemask = (uint)mask;
  return 0;
}
```

`argint(0, &mask)` 取出第一个参数，再通过 `myproc()` 找到当前进程，把掩码保存下来，最后返回 `0`。这一版 xv6 的 `argint()` 返回类型是 `void`，直接调用即可。

注册处理函数并输出跟踪信息

在 `kernel/syscall.c` 中声明 `sys_trace()`，并在系统调用分发表 `syscalls[]` 中添加对应项：

```
extern uint64 sys_trace(void);

// syscalls[] 中新增：
[SYS_trace]   sys_trace,
```

`syscallnames[]` 用系统调用编号作下标，保存打印时需要的名称。下面列出其中几项：

```
static const char *syscallnames[] = {
  [SYS_fork]  "fork",
  [SYS_read]  "read",
  [SYS_exec]  "exec",
  // 其余条目对应其他已注册的系统调用。
  [SYS_trace] "trace",
};
```

在 `syscall()` 的合法系统调用分支中，执行处理函数后检查掩码：

```
p->trapframe->a0 = syscalls[num]();

if(p->tracemask & (1U << num))
  printf("%d: syscall %s -> %d\n",
         p->pid, syscallnames[num], (int)p->trapframe->a0);
```

`num` 来自保存的 `a7`。`1U << num` 取出对应的位，与掩码按位与后，非零就打印。判断放在处理函数返回之后，此时 `a0` 已经保存了返回值；转成 `int` 打印，失败时就能显示为 `-1`。

测试结果

执行 `make qemu` 启动 xv6，然后在 shell 中测试。

只跟踪 `read`

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 961
3: syscall read -> 321
3: syscall read -> 0
```

输出里只有 `read`，与掩码 `32` 的设置一致。前三次返回值是读到的字节数，最后的 `0` 表示文件已读完。`grep` 的缓冲区大小是 1024 字节，但还要留出字符串结束符和未处理的行片段，所以每次读取的长度不完全相同。

`grep hello README` 查找文件中包含 `hello` 的行。这里没有匹配行，因此没有文本输出，但仍然能看到读取文件的调用记录。

跟踪当前所有系统调用

```
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 961
4: syscall read -> 321
4: syscall read -> 0
4: syscall close -> 0
```

`2147483647` 等于 `0x7fffffff`，第 0 ~ 30 位都是 `1`，包含目前所有系统调用的编号。

- `trace -> 0` 表示设置成功。返回后使用的是新掩码，所以这次 `trace` 自身也被打印出来。

- `exec -> 3` 是新程序的参数个数：`grep`、`hello`、`README`，共三个。本版本的 `exec()` 用返回值设置新程序的 `a0`，执行成功后继续运行的是 `grep`。
- `open -> 3` 表示打开了 `README`，文件描述符是 `3`。
- 随后的 `read` 逐步读取文件，直到返回 `0`；`close -> 0` 表示关闭文件成功。

输出中没有 `exit`，因为成功的 `exit()` 不会返回到打印代码所在的位置。

自动测试

Part A 的自动测试使用仓库自带的脚本：

```
./grade-lab-syscall trace
```

结果如下：

```
trace 32 grep: OK
trace all grep: OK
trace nothing: OK
trace children: OK
```

四项测试都通过了，分别检查指定调用、全部调用、未开启跟踪的情况和子进程继承。

遇到的问题及解决方法

测试时，我输入 `grep hello README` 后没有看到输出，一开始以为程序没有正常执行。后来确认，`README` 中没有包含 `hello` 的行，`grep` 自然不会打印匹配结果。加上 `trace 32` 后能看到四次 `read` 的记录，说明文件确实被读取了。

Part B

实现思路

Part B 的 `sysinfo()` 返回空闲物理内存的字节数，以及状态不为 `UNUSED` 的进程数量。课程还要求保存并打印学号，我在 `struct sysinfo` 中直接加入 `student_id`，成功查询时输出 `my student number is 24302010005`。

调用入口的添加方式与 Part A 相同。区别在于，这次用户程序传入的是 `sysinfo(&info)` 中的地址。内核先把结果填到自己的局部结构体里，再用 `copyout()` 写回用户空间。

统计空闲内存时遍历 `kmem.freelist`，每个节点算一页，也就是 `PGSIZE` 字节。遍历期间持有 `kmem.lock`。进程数量则逐个检查进程表项，在对应的锁下读取状态，睡眠进程和僵尸进程也要计入。

关键代码

定义返回结构体并补齐调用入口

在 `kernel/sysinfo.h` 中定义：

```
struct sysinfo {
  uint64 freemem;
  uint64 nproc;
  char student_id[12];
};
```

学号有 11 位，数组开 12 个字节，最后一个用于字符串结束符 `\0`。

在 `user/user.h` 中添加前向声明和函数原型：

```
struct sysinfo;
int sysinfo(struct sysinfo *);
```

函数参数是结构体指针，前向声明就够了。用户程序要访问具体字段时，再包含 `kernel/sysinfo.h`。

`user/usys.pl` 加入 `entry("sysinfo");`，`kernel/syscall.h` 分配编号 `23`。然后在 `kernel/syscall.c` 中声明 `sys_sysinfo()`，把它加入分发表，并在名称数组中加入 `"sysinfo"`，这样也能用 `trace` 跟踪它。测试程序通过 `Makefile` 中的 `$U/_sysinfotest` 加入 xv6 文件系统。

统计空闲内存

在 `kernel/kalloc.c` 中新增：

```
uint64
freemem(void)
{
  struct run *r;
  uint64 bytes = 0;

  acquire(&kmem.lock);
  for(r = kmem.freelist; r; r = r->next)
    bytes += PGSIZE;
  release(&kmem.lock);

  return bytes;
}
```

每个空闲节点代表 4096 字节，所以遍历时累加 `PGSIZE`。`kalloc()` 和 `kfree()` 都会修改这条链表，统计时也要持有同一把锁。

统计进程数量

在 `kernel/proc.c` 中新增：

```
uint64
nproc(void)
{
  struct proc *p;
  uint64 count = 0;

  for(p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);
    if(p->state != UNUSED)
      count++;
```

```
    release(&p->lock);
  }

  return count;
}
```

进程退出后先成为 `ZOMBIE`，此时还要计数。等父进程用 `wait()` 回收，表项变为 `UNUSED`，才从统计中排除。代码每次只锁一个表项，因此得到的是遍历期间的统计值，而不是整个进程表在同一时刻的快照。

两个统计函数都在 `kernel/defs.h` 中声明，供 `sys_sysinfo()` 调用。

实现系统调用处理函数

在 `kernel/sysproc.c` 中包含 `sysinfo.h`，并新增：

```
uint64
sys_sysinfo(void)
{
  uint64 addr;
  struct sysinfo info = {0};
  struct proc *p = myproc();

  argaddr(0, &addr);
  info.freemem = freemem();
  info.nproc = nproc();
  safestrcpy(info.student_id, "24302010005",
             sizeof(info.student_id));

  if(copyout(p->pagetable, addr, (char *)&info, sizeof(info)) < 0)
    return -1;

  printf("my student number is %s\n", info.student_id);
  return 0;
}
```

`argaddr()` 取出用户传入的地址，它本身不检查地址是否有效。统计结果和学号先写入内核里的 `info`，然后交给 `copyout()`，通过当前进程的页表复制到用户地址。复制失败返回 `-1`；成功才打印学号并返回 `0`。

`info` 初始化为零，避免把未初始化的内容带到用户空间。复制学号时用 `safestrcpy()`，保证末尾有字符串结束符。

测试结果

重新启动 xv6 后，在 shell 中运行 `sysinfotest`，输出如下：

```
$ sysinfotest
sysinfotest: start
my student number is 24302010005
my student number is 24302010005
my student number is 24302010005
my student number is 24302010005
my student number is 24302010005
my student number is 24302010005
my student number is 24302010005
my student number is 24302010005
sysinfotest: OK
```

学号打印了多次，是因为测试中反复调用了 `sysinfo()`。最后出现 `sysinfotest: OK`。

评分脚本中的 `sysinfotest` 项也通过了。完成 Part B 后又运行了 Part A 的四项测试，结果都是 `OK`。

实验问答

（1）System calls Part A 部分，简单描述一下 trace 全流程。

在 shell 中输入 `trace <mask> <command>` 后，shell 创建进程并运行 `trace` 程序。它将掩码参数转成整数，调用 `trace(mask)`。

汇编入口把 `SYS_trace` 放进 `a7`，参数在 `a0` 中，随后执行 `ecall`。陷入入口保存寄存器，再进入 `usertrap()`；后者识别出系统调用，调用 `syscall()`。分发函数读取保存的 `a7`，找到 `sys_trace()`，将参数写入当前进程的 `tracemask` 并返回 `0`。

返回值保存在 `trapframe->a0` 中。内核检查掩码，需要跟踪就打印，然后返回用户态。`trace` 程序接着用 `exec()` 运行指定命令，进程中的掩码保留下来。此后，被掩码选中的系统调用都会在返回时输出记录。`fork()` 会复制掩码给子进程，其他进程的设置不变。

（2）kernel/syscall.h 是干什么的，如何起作用的?

`kernel/syscall.h` 定义系统调用编号，例如 `SYS_read` 是 `5`，`SYS_trace` 是 `22`。它让用户态和内核使用同一套编号，具体处理函数写在其他源文件中。

生成的 `user/usys.S` 包含这个头文件，把对应的 `SYS_*` 编号放进 `a7` 后执行 `ecall`。内核的 `kernel/syscall.c` 同样包含它，使用编号建立 `syscalls[]` 数组，再根据 `a7` 选择处理函数。

Part A 还用这些编号确定掩码中的位和名称数组的下标。新增系统调用时，除了分配编号，还要添加用户态入口和分发表条目。只修改 `syscall.h`，调用仍然无法执行。

实验感想

做完这两个部分后，`a7` 和 `a0` 的用途更清楚了：前者告诉内核要执行哪个系统调用，后者既传递第一个参数，也保存返回值。`trace` 把这些调用直接打印出来，比只看函数声明更容易理解程序实际执行了什么。

Part B 中，用户传进来的是地址，却不能在内核里直接按普通指针使用，需要通过页表复制结果。统计进程时也要留意状态，退出并不等于立即从进程表中消失，僵尸进程仍要等 `wait()` 回收。